MD5 (Message Digest Algorithm 5)

1. Overview

MD5 is a cryptographic hash function designed by **Ronald Rivest in 1991**, producing a **128-bit (32 hexadecimal character) digest**.

It was once the dominant hash function for integrity checking and password storage, but is now **cryptographically broken** and must not be used for any security-relevant purpose.

MD5 at a Glance

Property	Detail
Type	Cryptographic hash function
Digest size	128 bits (32 hex characters)
Designed	1991, by Ronald Rivest
Current status	**Broken** — collision-resistance defeated since 2004

2. Why MD5 Is Broken

MD5's core security failure is in **collision resistance**.

Researchers demonstrated in **2004** (and dramatically improved since) practical, fast methods to construct two *different* inputs that produce the *identical MD5 hash*.

This directly violates one of the fundamental required properties of a secure hash function (see the SHA guide, Section 2) and means MD5 can no longer be trusted anywhere collision resistance matters.

Example Impact

An attacker could craft two different documents:

- One legitimate
- One malicious

while making both produce the same MD5 digest.

This can defeat an integrity or signature scheme relying on the assumption that a digest is unique to its content.

> **Key point:** MD5's collision resistance is considered broken and must not be relied upon for security.

---

3. Two Separate Problems: Broken Collision Resistance vs. Sheer Speed

It is important to distinguish MD5's two distinct weaknesses because they affect different use cases.

3.1 Collision Attacks

Collision attacks are the specific, proven mathematical break.

They matter most for:

- Digital signature forgery
- Integrity-verification scenarios
- Attacker substitution of content without changing the "verified" hash

3.2 Raw Computational Speed

MD5 is extremely fast to compute — **tens of billions of hashes per second can be achievable on modern GPU hardware**.

This is a separate problem from the collision break.

It is exactly what makes MD5, and even uncompromised fast hashes such as SHA-256, fundamentally unsuitable for password storage:

> An attacker with a dumped MD5 password hash can brute-force enormous numbers of candidate passwords per second, regardless of whether collision resistance is intact.

The Two Weaknesses at a Glance

Weakness	Primary Concern
Collision resistance failure	Signature forgery and adversarial integrity verification

Extreme computational speed	Fast password-hash brute-force attacks

4. Where MD5 Is Still Encountered (and Why It Shouldn't Be)

Despite being broken, MD5 remains widely present in legacy systems due to historical inertia.

Common examples include:

- **Old or poorly maintained applications** still hashing passwords with unsalted or salted MD5.
- **Legacy checksums for non-security-critical purposes**, such as quick file-corruption detection where an adversarial attacker is not a realistic threat model.
    - Even here, **SHA-256 is now generally preferred**, simply because there is no longer a compelling reason to reach for MD5.
- **Older protocol implementations and file formats** that specified MD5 before its weaknesses were widely known.

> **Important:** Finding MD5 in a legacy system does not automatically mean it is being used for the same purpose everywhere; its security implications depend on how it is being used.

5. What MD5 Should Never Be Used For

❌ Password Storage

MD5 is **trivially fast to brute-force** and lacks the deliberate slowness and memory-hardness that dedicated password-hashing functions provide.

Use dedicated password-hashing functions such as:

- **Argon2**
- **bcrypt**
- **PBKDF2**

❌ Digital Signatures or Certificate Signing

Collision attacks can be leveraged to forge signed content that appears to match a legitimately signed hash.

❌ Adversarial Integrity-Checking Contexts

MD5 should not be used anywhere an attacker might deliberately try to substitute malicious content while preserving a matching hash.

---

6. Remediation

Systems still relying on MD5 for password storage or integrity verification should migrate to modern alternatives.

Password Storage

Migrate to:

- **Argon2id**
- **bcrypt**
- **PBKDF2**

General-Purpose Integrity / Signature Use

Migrate to:

- **SHA-256**
- **SHA-3**
- Another current member of the **SHA-2/SHA-3 family**

Transparent Re-Hashing at Login

Password-storage migration can often be handled through **transparent re-hashing at login**:

```
User Login
    |
    ▼
Verify against existing MD5 hash
    |
    ▼
Authentication succeeds
    |
```

```
    ▼
Immediately hash the plaintext password
with a modern password-hashing algorithm
    |
    ▼
Update the stored password hash
```

This avoids requiring a disruptive mass password reset.

> **Important:** The old MD5 hash is used only for the final verification during migration, after which the stored value is replaced with a modern password hash.

---

7. Summary

MD5 is a legacy hash function that should be considered **fully obsolete for any security purpose**.

Its:

1. **Collision resistance is proven broken**
2. **Raw computational speed makes it categorically unsuitable for password storage**, regardless of collision status

Its continued presence in real-world systems is almost always a **legacy artifact** rather than a deliberate current choice.

> **Bottom line:** MD5 should be replaced wherever it is found in security-relevant use, with the replacement treated as a priority.

Quick Reference

Use Case	MD5 Status	Recommended Direction
Password storage	❌ Never use	Argon2id / bcrypt / PBKDF2
Digital signatures	❌ Never use	SHA-256 / SHA-3 with appropriate signature scheme
Adversarial integrity checks	❌ Never use	SHA-256 / SHA-3
Legacy non-security checksums	⚠️ Legacy only	Prefer SHA-256

Modern security applications	❌ Obsolete	Use current cryptographic algorithms